

StableGold Audit



August 19, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	4
Scope	5
System Overview	6
Security Model and Trust Assumptions	7
Privileged Roles	9
High Severity	10
H-01 Freeze Controls Can Be Bypassed Via burn, burnFrom, And Frozen Spenders In transferFrom	10
Medium Severity	11
M-01 Pooled Multi-Stablecoin Reserves Enable Depeg Arbitrage Through onchainBuyBack	11
M-02 buy And onchainBuyBack Are Unusable For Tokens That Omit ERC-20 Return Values	13
M-03 Cross-Chain Mint and Burn Bypass freezeList and kycStatus Compliance Controls	14
Low Severity	15
L-01 Redemption Is Bricked Until maxAmountforRedeem Is Configured	15
L-02 Constructor Does Not Verify That premintSupply Does Not Exceed maxSupply	15
L-03 admin Role Is a De Facto Superset of authority, custody, and minter	16
L-04 Configuration Setters Emit Events Inconsistently and Without Change Guards	17
L-05 Burn Paths Lack notPaused And Can Destroy Supply While Paused	18
L-06 Hardcoded Decimal Handling in buy Assumes an Unenforced Off-Chain Normalization	19
L-07 Zero-Valued Oracle Heartbeats Enforce Same-Block Updates And Can Block Core Flows	20
L-08 onchainBuyBack Lacks Minimum-Amount Check, Allowing Dust Sale to Forfeit Input for Zero Payout	20
L-09 Public encodeData And recoverSigner Helpers Diverge From On-Chain EIP-3009 Verification	21
L-10 Tokens Are Minted in buy Before Payment Is Pulled and Amount Is Verified	22
Notes & Additional Information	23
N-01 Inconsistent Use of msg.sender and _msgSender	23
N-02 Events Emitted Inside Loops Increase Gas Costs	23
N-03 Incorrect Order of Function Modifiers	24
N-04 Missing Security Contact	24
N-05 Missing Docstrings	24
N-06 Missing Named Parameters in Mappings	24
N-07 Missing Error Message in Revert Statement	25
N-08 Non-Explicit Imports Are Used	25
N-09 onChainBBLimit Functions as a Lifetime Cap Rather Than a Periodic Rate Limit	25
N-10 Ownership Transfer Is Single-Step and Can Irrecoverably Lose Control of the Contract	26

N-11 View Functions retrieveGoldPrices and retrieveOnChainGoldBBPrices Return Prices Without a Staleness Check	26
N-12 collectedBBFees Tracks Notional Margin Instead of Segregated Funds	27
N-13 Hardcoded EIP-712 Domain Name and Version Can Diverge from Token Metadata and Break Signed Transfers	28
N-14 Integrators Using transferWithAuthorization in Contract Flows Are Exposed to Front-Running	28
N-15 Multiple Contract Declarations per File	29
N-16 Token Amount Calculation in buy Divides Before Multiplying	30
N-17 Price and Reserve Feeds Use Redundant Chainlink-Shaped Interfaces	30
N-18 Proof-of-Reserve Can Be Enabled With a Zero Feed Address, Silently Falling Back to maxSupply Bound	31
Conclusion	32
Appendix	34
Issue Classification	34

Summary

Type	Stablecoin	Total Issues	32 (20 resolved)
Timeline	From 2026-07-29 To 2026-08-04	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	1 (1 resolved)
		Medium Severity Issues	3 (3 resolved)
		Low Severity Issues	10 (7 resolved)
		Notes & Additional Information	18 (9 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the [Stablegeneration/stablegen-token-contracts](#) repository at the [22283f5cf75d75a76f38731e147782aaef388e18](#) commit.

In scope were the following files:

```
contracts
└─ StableGold.sol
```

Update: *The fixes for the findings highlighted in this report have all been merged at commit [bcd20cb](#).*

System Overview

StableGen's StableGold (`SGOLD`) is a gold-backed asset-reference token: an 18-decimal ERC-20 whose units represent grams of physically held gold. Users mint `SGOLD` by paying an accepted stablecoin at a live gold price plus a premium, redeem it for physical gold or an over-the-counter buyback, or sell it back on-chain at the gold price minus a fee. On top of a standard ERC-20 core, the token layers oracle-driven pricing, role-based administration, compliance controls (KYC allow-listing and address freezing), a supply cap optionally reinforced by an on-chain proof-of-reserve feed, gas-abstracted signed transfers (EIP-3009), and a trusted-bridge cross-chain mint/burn interface (ERC-7802).

StableGold is a concrete instance of a parameterized design: the asset's name, symbol, price feed, initial supply cap, premium, buyback fee, accepted stablecoins, and redemption bounds are supplied at deployment or through privileged setters rather than hardcoded, so the same contract can back other assets, although the fixed troy-ounce-to-gram conversion specializes its pricing to precious-metal feeds. Deployment alone does not make an instance operative: the constructor sets only a subset of these values and leaves the contract paused with the deployer as the initial `admin`, so the operator must still register accepted stablecoins, set the fees and redemption bounds, grant the operational roles, and unpause before any user-facing flow can run.

The contract exposes a small set of user-facing and privileged flows. Users acquire `SGOLD` through `buy`, paying an accepted stablecoin, or receive operational mints through `mint`; they exit either through `redeem`, which burns the tokens or routes them to a dead address for physical or over-the-counter settlement, or through `onchainBuyBack`, which pays stablecoins back at the gold price minus a fee. A designated bridge moves supply across chains through `crosschainMint` / `crosschainBurn` (ERC-7802), and holders can delegate gas-abstracted transfers through the EIP-3009 `transferWithAuthorization` / `receiveWithAuthorization` entry points. Overlaid on all of these are the compliance and emergency controls (a KYC allow-list, a freeze block-list, and a global pause), alongside owner-only recovery paths (`reclaim`, `burnFreezedAssets`) that act on frozen balances.

Issuance is priced from the gold oracle, with a troy-ounce feed answer converted to a per-gram price (via `answer * 1e7 / 311034768`) before the premium is applied. Two distinct feeds are used: `priceFeed` supplies the gold spot price for buy and buyback math, while the separate `chainReserveFeed` supplies proof-of-reserve data used only to bound minting.

Minting is capped either by an admin-set `maxSupply` or, when proof-of-reserve is explicitly enabled and a non-zero feed is configured, by the live reserve figure, which is an either/or branch rather than a layered check. For off-chain quoting, the contract also exposes two view helpers, `retrieveGoldPrices` and `retrieveOnChainGoldBBPrices`, which return the raw feed price, the derived per-gram price, and the buy-side and sell-side prices, whose difference is the protocol spread (premium plus buyback fee).

Security Model and Trust Assumptions

The following assumptions are load-bearing for the security of the system. They are not defects in themselves, but the guarantees of the token degrade or fail if any of them does not hold in the deployed configuration.

- **The backing invariant rests on trusted attestations, not a trustless proof of reserves.** `SGOLD` issuance is bounded on-chain by the admin-set `maxSupply` or, when proof-of-reserve is enabled with a non-zero feed, by a live reserve figure, in both cases checked against `totalSupply`. That mode adds real on-chain enforcement, but only against the number the reserve feed reports; nothing on-chain measures real vault gold, so full backing still rests on the operators, the reserve-feed operator, and the off-chain custody and audit process.
- **Economically retired supply is cleared operationally, not on-chain.** Tokens retired through non-burn redemption (routed to the dead address when `burnRedeem` is `false`) and through `onchainBuyBack` and over-the-counter redemption (held by the contract) are not burned and remain counted in `totalSupply`, so `totalSupply` reflects existing rather than circulating supply, and the retired portion continues to occupy the active `maxSupply` or proof-of-reserve mint bound until cleared. Continued availability of `buy`, `mint`, and `crosschainMint` therefore depends on the operator periodically burning or reselling parked buyback balances and, when `burnRedeem` is `false`, on raising `maxSupply` as permanently unburnable dead-address (`0xdEaD`) balances accumulate. Consumers that need circulating supply, such as explorers, exchanges, and backing-ratio or proof-of-reserve reporting, must compute it as `totalSupply` minus the dead-address and contract balances.
- **Privileged roles are separable by design but can collapse in practice.** The `owner`, `admin`, `authority`, `custody`, `minter`, and `tokenBridge` roles are distinct, but

`admin` is a de facto superset of `authority` / `custody` / `minter`. Separation of duties holds only if each role is a distinct, appropriately controlled principal (`owner` a Safe multisig and `admin` a tightly controlled multisig rather than a single-key EOA).

Otherwise, a single compromised operational key unifies compliance, KYC, and minting.

- **Cross-chain authority is delegated wholesale to a single trusted bridge.** Per ERC-7802, `crosschainMint` / `crosschainBurn` are gated only by `onlyTokenBridge`, so when `crossChainStatus` is enabled the single `tokenBridge` address can mint up to the remaining cap to any destination and burn any holder's balance (the burn path enforces no freeze check and ignores `pause`). It is one address with no per-bridge cap and is repointable instantly via `setTokenBridge`. Safety therefore rests on the owner pointing it only at a vetted bridge contract, and on that bridge's own key and upgrade model, neither of which the token enforces on-chain.
- **`acceptedTokens` holds only well-behaved stablecoins.** Payment and payout math assumes every accepted token is a plain ERC-20 with no transfer fee, no rebasing, no hooks or callbacks, and exactly 6 or 18 decimals matching the off-chain 18-decimal `amount` convention. A non-conforming token would break collateral accounting or add a reentrancy surface, so curating this list is a standing operational responsibility.
- **Trust-critical sources can be repointed instantly, with no timelock.** `updatePriceFeed`, `setChainReserveFeed`, and `setTokenBridge` each change the pricing source, reserve source, or bridge authority in a single owner transaction with no delay, so "who is the oracle or bridge" can change silently between blocks and monitoring cannot rely on advance notice.
- **Payout liquidity exists only at the owner's discretion.** `withdrawERC20` is an unrestricted owner sweep of any held token, including the stablecoins that fund `onchainBuyBack` payouts and the principal paid through `buy`. Nothing segregates payout liquidity, so any "always redeemable on-chain" guarantee holds only while the owner leaves sufficient balance in the contract.
- **Compliance gates who acts, not where value ultimately settles.** KYC and freeze bind the party initiating each flow rather than the final destination: `buy` and `onchainBuyBack` mint or pay to a caller-chosen `_to` that is not checked against the lists, and both `SGOLD` and the payout stablecoins remain freely transferable once received. Enforcing the ultimate destination of funds is therefore an off-chain responsibility rather than an on-chain guarantee.
- **Secure operation depends on correct post-deployment configuration.** The contract neither validates nor atomically initializes its configuration, several parameters are unsafe at their zero defaults or behave counter-intuitively, and a number of privileged actions emit no event or are irreversible. Correct, complete, and consistent operator setup is

therefore assumed, with the specific misconfiguration and monitoring risks detailed in the findings.

Privileged Roles

Access control follows a two-tier model. `Ownable` provides the `owner` (intended to be a Safe multisig) for the most sensitive operations, while a set of boolean mappings governs operational actions. Each non-owner modifier also admits `admin`, making `admin` an operational superset of the other roles.

- **Owner** (`Ownable`, intended Safe multisig): `pauseStatus`, `increaseSupply` / `decreaseSupply`, `reclaim`, `burnFreezedAssets`, `updatePriceFeed`, `setChainReserveFeed`, `setTokenBridge`, `addAdmin`, `addAcceptedStables`, `updateBuyBackAddress`, `withdrawERC20` / `withdrawCollectedFees`, and all feature toggles.
- **admin**: grants the `minter` / `authority` / `custody` roles, sets `premium` and `onchainBuyBackFee`, and, by virtue of the modifier design, can perform any `authority` / `custody` / `minter` action.
- **authority**: `freezeAddress` and `batchFreezeAddresses` (the compliance block-list).
- **custody**: `updateKYCStatus` and `updateKYCAddressBatch` (the KYC allow-list and on-chain buyback limits).
- **minter**: `mint`.
- **tokenBridge**: `crosschainMint` and `crosschainBurn`.

On-chain details of the EOAs and multisigs backing these roles should be confirmed against the deployed configuration.

High Severity

H-01 Freeze Controls Can Be Bypassed Via `burn`, `burnFrom`, And Frozen Spenders In `transferFrom`

`StableGold` implements a compliance freeze through the `freezeList` mapping, set by an authority via `freezeAddress`, and provides owner recovery paths in `reclaim` and `burnFreezedAssets`. Freeze enforcement is implemented only in the `transfer` and `transferFrom` overrides, each of which checks `freezeList[from]` and `freezeList[to]`.

This enforcement is incomplete in two respects. First, `transferFrom` checks only the `from` and `to` endpoints and never the spender, so a frozen address that holds an allowance granted before it was frozen can continue to move a third party's tokens between two non-frozen accounts. This does not let a frozen address move its own frozen balance, since that balance would sit in the checked `from` position and cause the call to revert; the gap is instead that a frozen party retains the ability to act on other holders' balances that it was previously approved to spend, an activity the freeze is presumably meant to suspend. Second, `burn` and `burnFrom` are inherited from `ERC20Burnable` and carry no freeze check, and the `_beforeTokenTransfer` hook is empty, so no freeze policy backstops them at the token level. A frozen holder can therefore call `burn` to destroy the exact balance the owner intends to recover through `reclaim`, and a spender can `burnFrom` a holder regardless of freeze status. Both recovery paths are owner-only and act on the balance present at call time, `reclaim` through `_transfer` and `burnFreezedAssets` through `_burn`, and neither snapshots or locks the balance at freeze time. Because the freeze is set by an `authority` and recovery must then be executed in a separate owner transaction, while the frozen holder's `burn` is permissionless and takes effect immediately, a holder who burns first leaves `reclaim` and `burnFreezedAssets` with nothing to recover.

Enforcement is likewise absent from issuance: neither `mint` nor `crosschainMint` checks the recipient against `freezeList`, so tokens can be minted to a frozen address. Minting to an address that is itself frozen is a consistency gap rather than an exfiltration path, because that recipient cannot subsequently transfer the tokens; the separate scenario in which the bridge mints to a fresh, non-frozen address to complete a cross-chain escape is reported as its own finding. Either way, the freeze policy is applied unevenly across the supply-changing entry points. The consequence is that freezing does not reliably immobilize a targeted balance,

which undermines both the freeze as a compliance control and the recovery flow that depends on it.

Consider first confirming the intended threat model for the freeze. The combination of an `authority`-set block-list with owner clawback through `reclaim` and `burnFreezedAssets` is characteristic of a regulatory or compliance block-list, in which a targeted balance is expected to remain both immobilized and recoverable, rather than a voluntary user self-freeze; under that model, permitting a frozen party to burn its own tokens or to keep using allowances granted before the freeze defeats the control. If a frozen address is meant to be immobilized entirely, including being prevented from burning its own tokens, enforce the freeze policy centrally so that it applies uniformly to mints, burns, and transfers, for example within `_beforeTokenTransfer`, adding a spender check to `transferFrom`, and overriding `burn` and `burnFrom` to revert when the affected account is frozen. If instead only transfers are intended to be blocked, align the enforcement to that narrower policy. Consider also blocking or revoking allowances that involve frozen accounts so that approvals granted before a freeze cannot be used afterwards. The same central enforcement would also close the related cross-chain compliance gap in `crosschainMint` and `crosschainBurn`, which is reported separately as "Cross-Chain Mint and Burn Bypass" `freezeList` and `kycStatus` Compliance Controls" because its exploitability additionally depends on the external bridge.

Update: Resolved in [pull request #2](#) at commit [64f7a24](#). The team stated:

Freeze policy was added to specific functions. We have also override `burn` and `burnFrom` to check the freeze policy.

Medium Severity

M-01 Pooled Multi-Stablecoin Reserves Enable Depeg Arbitrage Through `onchainBuyBack`

`StableGold` accepts several stablecoins as payment through the owner-managed `acceptedTokens` allowlist. The `buy` path mints `SGOLD` from the gold oracle price and the caller-supplied `amount`, using `_token` only for the allowlist and `decimals` checks. The `amount` argument is interpreted as an 18-decimal value quantity that is independent of `_token`: the number of tokens minted is computed as `amount / goldPricePremium * 1e8`, and `_token` affects only how many raw units are pulled from the caller, namely `amount` for 18-decimal tokens and `amount / 1e12` for 6-decimal tokens. An identical `amount`

therefore mints identical `SGOLD` regardless of which accepted stablecoin is paid. There is no per-token price, no peg verification, and no segregation of the reserves accumulated per stablecoin.

Because `onchainBuyBack` allows the caller to select any accepted `_token` as the payout asset, the contract effectively pools every accepted stablecoin into a single reserve and treats each as worth exactly one unit of account. Access to this path is gated: the caller must be KYC-approved (`kycStatus`), must not be frozen, and the `onchainbuyBackStatus` toggle must be enabled, with per-account throughput bounded by a custody-assigned `onChainBBLimit` and no time lock. The `buy` leg is open to any caller when `saleStatus` is enabled and otherwise also requires KYC and `custody` approval. Within these constraints, if any accepted stablecoin trades materially below its peg while remaining on the allowlist, an eligible actor can acquire the weak stablecoin cheaply, mint `SGOLD` at par through `buy`, and redeem through `onchainBuyBack` while selecting the strongest stablecoin the contract holds. Both legs price off a fixed oracle read with no price-impact or slippage, so the only frictions are the `premium` charged on `buy` and the `onchainBuyBackFee` deducted from the payout, and the loop is profitable whenever the depeg discount on the acquired stablecoin exceeds the sum of these two fees. This drains the most valuable reserve and leaves the depegged asset behind, transferring the loss to the protocol and to the remaining holders. The per-address `onChainBBLimit` bounds throughput per account but does not prevent the drain, since eligibility can be distributed across multiple approved accounts.

Consider segregating reserves and liabilities per accepted stablecoin so that amounts minted against one token can only be redeemed in the same token, or pricing each accepted stablecoin against a reliable market feed so that minting and redemption reflect current exchange rates. Consider also limiting `acceptedTokens` to a single canonical settlement asset, or routing redemptions through a managed swap into that asset, so that a depeg of one accepted token cannot drain the reserves held in another.

Update: Resolved. After further discussion with the client, it is noted that buys and on-chain buybacks are never enabled simultaneously: buys are restricted to a single designated stablecoin, and on-chain buybacks are enabled only once the total supply is fully minted and sales have stopped, accepting only a single designated token and capped at a specific total amount.

M-02 `buy` And `onchainBuyBack` Are Unusable For Tokens That Omit ERC-20 Return Values

The contract interacts with accepted stablecoins through the `IERC20` interface, whose `transfer`, `transferFrom`, and `approve` functions are declared to return a boolean. When Solidity performs an external call to a function declared with a return value, it validates that the callee returned at least the expected number of bytes and reverts during return-data decoding if it did not. Several widely used tokens, most notably USDT, do not conform to this part of the ERC-20 interface and declare no return value on `transfer`, `transferFrom`, and `approve`. A call to such a token through a `returns (bool)` interface therefore reverts on decoding even when the underlying operation would have succeeded.

This behavior makes the core flows unusable for such tokens. In `buy`, the payment is collected through `bool success = IERC20(_token).transferFrom(...)`, which reverts on decoding for a USDT-style token, so users cannot mint `SGOLD` against it. The `onchainBuyBack` function pays the seller through the same pattern and reverts for the same reason. Furthermore, `approveTokenContract`, which must succeed for `onchainBuyBack` to move funds out of the contract, itself calls `IERC20(_token).approve(...)` and reverts on decoding, so the buyback flow cannot even be configured for such a token. Because `acceptedTokens` is an owner-curated list populated through `addAcceptedStables`, this condition arises only when a non-conforming token is deliberately added; however, USDT is the largest stablecoin by market capitalization and is therefore a likely candidate. The condition fails safe in that no funds are lost or locked, but the primary product functionality would be entirely unavailable for such a token.

The same raw `IERC20` pattern is used beyond the entry and exit points. Both `withdrawERC20` and `withdrawCollectedFees` move accepted tokens through `IERC20(...).transfer` without accommodating a missing boolean return, so a USDT-style token would additionally break owner withdrawals of contract balances and collected fees. Any remediation should therefore be applied uniformly across every path that handles accepted tokens, not only the issuance and buyback entry points.

Consider using a safe wrapper such as OpenZeppelin `SafeERC20`, with `safeTransferFrom`, `safeTransfer`, and `forceApprove`, for all external token interactions, including `buy`, `onchainBuyBack`, `approveTokenContract`, `withdrawERC20`, and `withdrawCollectedFees`. This tolerates tokens that omit the boolean return value as well as tokens that return `false` instead of reverting, and it allows USDT and similar assets to be supported consistently across the system.

Update: Resolved in [pull request #2](#) at commit [f156b26](#). The team stated:

| We have modified the code to use `SafeERC20`.

M-03 Cross-Chain Mint and Burn Bypass `freezeList` and `kycStatus` Compliance Controls

`StableGold` supports cross-chain movement through `crosschainMint` and `crosschainBurn`, both gated only by `onlyTokenBridge` and by `crossChainStatus`, an owner-controlled flag that enables the cross-chain feature globally and performs no per-address check. On-chain flows, by contrast, enforce compliance at the entry point: `transfer` and `transferFrom` require that neither endpoint is present in `freezeList`, and `kycStatus` is separately required on the caller of `redeem` and `onchainBuyBack`, as well as of `buy` while `saleStatus` is disabled.

Neither cross-chain hook applies these controls. `crosschainMint` mints to an arbitrary `_destination` and `crosschainBurn` burns from an arbitrary `_from` without checking `freezeList` or `kycStatus` on either address, and `crosschainBurn` additionally destroys a holder's balance with no allowance or signature requirement beyond trusting the bridge. If `tokenBridge` is a user-triggerable contract that does not itself enforce these lists, a frozen or non-compliant holder can burn on the source chain and receive freshly minted tokens at a new address on the destination chain, escaping a freeze that on-chain transfers would have blocked. The missing `kycStatus` check similarly lets the bridge credit `_destination` addresses that have not passed the allow-list enforced on the redemption and buyback paths. Because the outcome depends entirely on the external bridge implementation, the compliance control effectively rests on the bridge rather than on the token.

Consider enforcing `freezeList` and `kycStatus` on `_from` and `_destination` directly within `crosschainBurn` and `crosschainMint` if cross-chain movement is meant to be subject to the same compliance regime as on-chain transfers. If compliance is intentionally delegated to the bridge, consider documenting that trust boundary explicitly and constraining the `tokenBridge` address, which the `owner` sets and can repoint through `setTokenBridge`, to a vetted implementation, including its upgrade controls, that applies the required checks before invoking these hooks. Vetting that implementation and its upgrades then rests with whichever governance or compliance function controls the `owner` key, intended to be a Safe multisig. This is the cross-chain manifestation of the freeze-enforcement gap reported in "Freeze Controls Can Be Bypassed Via `burn`, `burnFrom`, And Frozen Spenders In `transferFrom`"; the central `_beforeTokenTransfer` enforcement

recommended there would also cover these hooks, leaving the `kycStatus` check and the bridge trust boundary specific to this finding.

Update: Resolved in [pull request #2](#) at commit [10ac16b](#). The team stated:

| We enforced the `freezeList` on crosschain functions.

Low Severity

L-01 Redemption Is Bricked Until `maxAmountforRedeem` Is Configured

The `redeem` function bounds the redeemed amount with `require(amount >= minAmountforRedeem && amount <= maxAmountforRedeem, ...)`. Both `minAmountforRedeem` and `maxAmountforRedeem` default to zero and are set only through `updateRedeemMaxMinAmount`.

Until the owner configures a non-zero `maxAmountforRedeem`, the bound reduces to `amount >= 0 && amount <= 0`, which permits only `amount == 0`. Every non-zero redemption therefore reverts, while a zero-amount call burns or transfers nothing yet still emits `redeemEvent`. Redemption is therefore unavailable until the owner explicitly sets the maximum, and the failure surfaces as a silent configuration-ordering dependency rather than an explicit error. The contract deploys paused and requires owner configuration before go-live, so this is likely to be caught during setup, which bounds the impact.

Consider initializing `maxAmountforRedeem` to a sensible non-zero value, or documenting the requirement to configure the redemption bounds before unpausing, so that redemption is not silently disabled by an unset maximum.

Update: Resolved in [pull request #2](#) at commit [7b226e5](#).

L-02 Constructor Does Not Verify That `premintSupply` Does Not Exceed `maxSupply`

The constructor `mints` `premintSupply` to the deployer and separately `sets` `maxSupply` from `_initialMaxSupply`, without requiring that the premint not exceed the cap. If the contract is deployed with `_premintSupply` greater than `_initialMaxSupply`,

`totalSupply` exceeds `maxSupply` from deployment. Every non-reserve mint path then evaluates `require(totalSupply() + amount <= maxSupply)` and reverts, so `buy`, `mint`, and `crosschainMint` are all blocked until the owner raises the cap through `increaseSupply`. The `decreaseSupply` function guards against lowering `maxSupply` below `totalSupply`, but that check runs only within `decreaseSupply` and does not cover the constructor.

This is a deployment-time misconfiguration that is owner-recoverable rather than an attacker-exploitable condition, but it silently disables the primary mint flows from the first block.

Consider adding a `require(_premintSupply <= _initialMaxSupply)` check to the constructor so that the invariant `totalSupply() <= maxSupply` holds from deployment.

Update: Resolved in [pull request #2](#) at commit [286a47c](#). The team stated:

| *We added the required statement on the constructor.*

L-03 `admin` Role Is a De Facto Superset of authority, custody, and `minter`

The contract defines four operational roles that suggest a separation of duties, but the `onlyAuthority`, `onlyCustody`, and `onlyMinter` modifiers each admit `admin` in addition to the dedicated role, through an `admin[msg.sender] == true || ...` condition. The `admin` role is a mapping populated by the owner through `addAdmin`, so more than one `admin` address may exist at once and each independently holds the full operational power described here. As a result, any single `admin` address can freeze addresses through `freezeAddress` and `batchFreezeAddresses`, set KYC status and per-account limits through `updateKYCStatus` and `updateKYCAddressBatch`, mint, and grant or revoke those same roles through `addMinter`, `addAuthority`, and `addCustody`. The intended separation holds only for dedicated, non-admin holders of each role; at the `admin` tier it collapses into one all-powerful operational principal, so compromise or misuse of any single `admin` key simultaneously compromises freezing, KYC administration, and minting.

This unification is bounded to the operational roles and does not extend to the owner. The `onlyOwner` modifier checks the `Ownable` owner independently of the `admin` mapping, so owner-tier powers such as pausing, `increaseSupply`, `reclaim`, `burnFrozenAssets`, and the feed, bridge, and feature setters are not reachable through `admin`. The genuine separation boundary is therefore owner versus admin, while below the owner the `admin` role unifies the rest. This is very likely an intentional convenience rather than a defect, but it should be documented as an explicit trust assumption, and the deployed `admin` key should be a

tightly controlled multisig rather than an externally owned account, given the concentration of compliance and economic powers.

Consider documenting the admin-unifies-operational-roles behavior as an explicit trust assumption, and, if genuine separation of duties is required, migrating to a role framework such as OpenZeppelin [AccessControl](#), where holding an administrative role does not implicitly satisfy another role's gate and each grant is mediated and emitted, so that compliance, minting, and role administration can be assigned to independent principals.

Update: Acknowledged, not resolved. The team stated:

Thanks for your recommendation, however we wish to stay on our own access-control framework. We will document accordingly.

L-04 Configuration Setters Emit Events Inconsistently and Without Change Guards

Emitting an event on every privileged configuration change is a best practice that allows off-chain monitoring, indexing, and incident response to observe changes to sensitive parameters. While such changes are recorded in contract state on-chain regardless, without an event they leave no structured, easily indexable log for monitoring to consume, and since these privileged setters are expected to be called infrequently the concern is visibility rather than event volume. In [StableGold](#), event coverage across the configuration surface is inconsistent in two distinct ways.

First, most state-changing owner and admin setters emit no event at all, so privileged changes leave no explicit event-based on-chain trace. A subset does emit, namely [freezeAddress](#) and [batchFreezeAddresses](#), [pauseStatus](#), [updateKYCStatus](#) and [updateKYCAddressBatch](#), and [setTokenBridge](#), but the following emit nothing:

- Role grants: [addAdmin](#), [addMinter](#), [addAuthority](#), and [addCustody](#), so privilege escalation is invisible to event-based monitoring.
- Trust-critical parameters: [updatePriceFeed](#), [setChainReserveFeed](#), [increaseSupply](#), [decreaseSupply](#), and [updateBuyBackAddress](#).
- Economic parameters: [updatePremium](#) and [updateOnChainBuyBackFee](#).
- Feature toggles: [updateSaleStatus](#), [updateRedeemStatus](#), [updateBurnRedeemStatus](#), [updateBuyBackStatus](#), [updateOnChainBuyBackStatus](#), [updateCrossChainStatus](#), and [updateSignedTxStatus](#).

- Other configuration: `updateRedeemMaxMinAmount`, `addAcceptedStables`, and `approveTokenContract`.

Second, several setters that do emit fire their event without first checking whether the stored value has actually changed, so the same value can be set repeatedly and identical events re-emitted. This is primarily a monitoring-noise concern, but it can also disrupt off-chain automation that assumes each event represents a distinct change. This occurs in `pauseStatus`, which sets `pause`, and `setTokenBridge`, which sets `tokenBridge`.

Consider emitting a dedicated event on every privileged state change, ideally recording the previous and new values, and guarding each setter so that it emits only when the stored value actually changes rather than on every call.

Update: Acknowledged, not resolved. The team stated:

We have added emit events only on necessary functions to save contract size, thanks for the recommendation we wish to keep it as is for simplicity.

L-05 Burn Paths Lack `notPaused` And Can Destroy Supply While Paused

The `notPaused` modifier is applied to the movement and privileged-mint entrypoints, including `transfer`, `transferFrom`, and `crosschainMint`, which indicates that pausing is intended as an emergency stop for token movement and supply changes. Several supply-destruction paths are nonetheless left callable while `pause` is `true`. The counterpart `crosschainBurn` omits `notPaused`, so the configured `tokenBridge` can continue to burn any holder's balance during a pause whenever `crossChainStatus` is enabled. In addition, the inherited `burn` and `burnFrom` from `ERC20Burnable` are not overridden and carry no pause check, so any holder can burn, and any spender with a pre-existing allowance can `burnFrom`, while the contract is paused.

The concern is that the emergency brake is incomplete and does not stop supply from being destroyed while the contract is paused.

Consider applying `notPaused` to `crosschainBurn` and overriding `burn` and `burnFrom` to apply it as well, so that pausing halts every supply-destruction path, or documenting an explicit rationale if any burn path is intended to remain available while the contract is paused.

Update: Resolved in [pull request #2](#) at commit [64f7a24](#). The team stated:

We have applied `notPaused` on `burn`, `burnFrom`, `crosschainMint`, and `crosschainBurn`.

L-06 Hardcoded Decimal Handling in `buy` Assumes an Unenforced Off-Chain Normalization

The `buy` function takes a caller-supplied `amount` parameter, reads the payment token's decimals, and [requires the result to be exactly 6 or 18](#), then pulls `amount` for an 18-decimal token and `amount / 1e12` for a 6-decimal token. The minted `noOfTokens` is computed as `amount / goldPricePremium * 1e8`, where `goldPricePremium` is the per-gram gold price plus premium, so it is priced against the full, untruncated `amount` in both branches while only the transferred payment is truncated. The scheme is consistent only when `amount` is expressed in 18 decimals and is a whole multiple of `1e12` for a 6-decimal token, a convention that is never enforced on-chain. When `amount` is not a multiple of `1e12`, the integer division `amount / 1e12` truncates downward while the mint remains priced on the full `amount`, so the protocol collects less stablecoin than the minted tokens are priced for, with the rounding always favoring the buyer. In the extreme, for `goldPricePremium <= amount < 1e12` the collected payment truncates to zero while a positive `noOfTokens` is still minted, since `buy` guards `require(noOfTokens > 0)` but never checks that the payment is positive. The per-call shortfall is bounded below `1e-6` of the settlement stablecoin, so the practical impact is dust-scale relative to gas, but the behavior is a correctness defect rather than the consistent handling the convention assumes.

Consider defining `amount` in the payment token's native units and normalizing arithmetically from the reported `decimals`, scaling a six-decimal `amount` up by `1e12` before computing `noOfTokens` and transferring the unscaled native `amount`, so that no division truncation occurs. This removes the underpayment at its source, which a positivity guard such as `require(amount / 1e12 > 0)` would not, since that check would only reject the zero-payment case while still allowing partial underpayment for any `amount` that is not a multiple of `1e12`.

Update: Resolved in [pull request #2](#) at commit [f017e65](#). The team stated:

Now each amount needs to be sent to the correct token's decimals. We are scaling the 6 decimal amount as recommended.

L-07 Zero-Valued Oracle Heartbeats Enforce Same-Block Updates And Can Block Core Flows

The oracle staleness guards take the form `require(block.timestamp - updatedAt <= heartbeat, ...)`, as in `buy` and `onchainBuyBack` for `dataFeedHeartbeat`, and in the proof-of-reserve branches of `buy`, `mint`, and `crosschainMint` for `chainReserveHeartbeat`. When the applicable heartbeat is zero, the condition reduces to requiring that `updatedAt` equal the current block timestamp, so the feed must have been updated in the same block or the call reverts. A zero value has no plausible legitimate use, since it would force every dependent call into the same block as a feed update, and is therefore treated here as a misconfiguration.

The constructor sets `dataFeedHeartbeat` directly from its parameter without requiring a non-zero value, and `updatePriceFeed` and `setChainReserveFeed` likewise accept zero. Furthermore, `chainReserveHeartbeat` is not set in the constructor at all and therefore starts at the default zero until `setChainReserveFeed` is called. Deploying or reconfiguring with a zero heartbeat silently blocks `buy`, `onchainBuyBack`, and, under proof-of-reserve, all minting in every block in which the relevant feed was not updated, producing a denial of service driven by a missing configuration value rather than by an attacker. The condition is owner-recoverable and, because the contract is deployed paused, likely to be noticed before launch.

Consider requiring a non-zero heartbeat, and optionally a sane upper bound to reject excessively stale prices, in the constructor and in both feed setters, so that a zero value cannot silently stall the price-dependent flows.

Update: Resolved in [pull request #2](#) at commit [6be5c28](#). The team stated:

| *We now enforce a non-zero heartbeat on the specific functions and constructor.*

L-08 `onchainBuyBack` Lacks Minimum-Amount Check, Allowing Dust Sale to Forfeit Input for Zero Payout

The `onchainBuyBack` function moves the seller's tokens to the contract through `transfer(address(this), _amount)` before the stablecoin payout is computed and, unlike `redeem`, it enforces neither a minimum amount nor a `require(noOfTokens > 0)` check. For a sufficiently small `_amount`, the payout `noOfTokens = _amount * buybackPrice / 100000000` is truncated to zero by the further `/ 1e12` division that is

applied last for a six-decimal stablecoin. In that case the seller's tokens are taken while no stablecoin is returned.

The payout arithmetic itself is correct and multiplies before dividing, so no precision is lost for ordinary amounts, and the affected amounts are economically negligible, below roughly `1.6e-8` token when paying out a six-decimal stablecoin. The forfeited tokens are transferred to the contract's own balance rather than burned, so they remain recoverable by the owner through `withdrawERC20`, making this a value asymmetry stranded pending owner action rather than a permanent loss. Even so, the user-facing path can take input and return nothing with no automatic remedy.

Consider adding a minimum-amount check and a `require(noOfTokens > 0)` guard to `onchainBuyBack`, mirroring the `minAmountforRedeem` and `maxAmountforRedeem` bounds already enforced on the `redeem` path, so that sub-threshold sales revert instead of forfeiting the seller's input.

Update: Resolved in [pull request #2](#) at commit [99b918e](#). The team stated:

We have added as recommended the `require(noOfTokens > 0)` guard, now dust-transactions fail.

L-09 Public `encodeData` And `recoverSigner` Helpers Diverge From On-Chain EIP-3009 Verification

`StableGold` verifies EIP-3009 authorizations through the inherited `EIP3009` logic, which builds the struct hash with `keccak256(abi.encode(...))` and validates it through `SignatureChecker.isValidSignatureNow`, which accepts both externally owned account and ERC-1271 contract-wallet signatures. Two public helpers intended for off-chain use diverge from this path: `recoverSigner` performs ECDSA-only recovery and cannot validate ERC-1271 signatures, and `encodeData` returns a first value built with `abi.encodePacked`, so hashing it as `keccak256(data)` does not reproduce the on-chain `abi.encode` struct hash because packed encoding does not left-pad addresses and other value types to 32 bytes. An integrator relying on these helpers for pre-validation can therefore reject a valid ERC-1271 signature, or accept a signature over `keccak256(data)` that reverts on submission and wastes relayer gas. These are auxiliary views that do not affect the security of the transfer path itself.

Consider removing these helpers from the production interface if they are not intended for integrators, or, if pre-validation is intended, exposing only utilities that mirror the on-chain path by returning the exact EIP-712 digest verified by `EIP3009` and validating signatures through `SignatureChecker` rather than ECDSA-only recovery. Consider also renaming `recoverSigner` so that its externally-owned-account-only behavior is explicit.

Update: Resolved in [pull request #2](#) at commit [80553c4](#). The team stated:

We have dropped the first value (data), the helper function now returns just the `abi.encode` value.

L-10 Tokens Are Minted in `buy` Before Payment Is Pulled and Amount Is Verified

The `buy` function mints gold tokens to the recipient by calling `_mint` before pulling the stablecoin payment through `transferFrom`. It trusts the caller-supplied `amount` and a truthy return value rather than comparing the contract balance before and after the transfer. If an accepted token charges a fee on transfer, is rebasing or deflationary, or is upgraded to behave that way, the contract would receive less than `amount` while the buyer receives gold priced against the full `amount`, under-collecting stablecoin relative to the tokens minted.

The likelihood is limited. The accepted-token set is controlled exclusively by the owner through the `onlyOwner`-gated `addAcceptedStables` function, mainstream stablecoins do not charge transfer fees (for example, `USDC` has no such mechanism and `USDT` retains only a dormant, capped one), and the mint remains bounded by `maxSupply` and the reserve feed.

Consider measuring the contract balance immediately before and after the `transferFrom` call and using the observed delta as the authoritative payment amount, reverting when it is insufficient for the tokens minted, and minting only after payment has been confirmed.

Update: Acknowledged, not resolved. The team stated:

The set of accepted stablecoins is controlled by us and therefore, we will only be adding mainstream stablecoins that do not charge transfer fees. If any token in the future introduces any transfer fee will be removed from the set.

Notes & Additional Information

N-01 Inconsistent Use of `msg.sender` and `_msgSender`

The contract inherits `Context` and therefore exposes `_msgSender`, yet the role modifiers such as `onlyAdmin` and the `buy`, `crosschainMint`, and `crosschainBurn` functions read `msg.sender` directly while other paths rely on `_msgSender`. The contract does not currently support meta-transactions, so this is not presently a defect, but mixing the two accessors is inconsistent and would become a correctness hazard if a `Context` override were ever introduced.

Consider using a single accessor consistently, preferably `_msgSender`, throughout the contract.

Update: Resolved in [pull request #2](#) at commit [b57e96](#). The team stated:

| *We standardized on a single accessor as recommended.*

N-02 Events Emitted Inside Loops Increase Gas Costs

The `batchFreezeAddresses` and `updateKYCAddressBatch` functions emit an event on every iteration of their loops. For large input arrays this increases gas costs relative to emitting a single aggregate event, although per-item events do assist off-chain indexing.

Consider emitting a single batch-level event for these operations, or confirming that per-item emission is an intentional trade-off made for off-chain indexing needs.

Update: Acknowledged, not resolved. The team stated:

| *This is intentional. Thanks.*

N-03 Incorrect Order of Function Modifiers

The `transfer` and `transferFrom` overrides declare their modifiers as `public notPaused virtual override`, placing the custom `notPaused` modifier before `virtual` and `override`. The Solidity style guide recommends the order visibility, mutability, `virtual`, `override`, and then custom modifiers.

Consider reordering the modifiers to follow the recommended convention.

Update: Resolved in [pull request #2](#) at commit [fa75faa](#). The team stated:

| *Thanks for the recommendation. We have made the recommended fix.*

N-04 Missing Security Contact

The contracts do not declare a security contact, which leaves no on-chain indication of how to report a vulnerability responsibly.

Consider adding a `@custom:security-contact` NatSpec entry above each contract definition so that vulnerability disclosure can be routed to the intended channel.

Update: Resolved in [pull request #2](#) at commit [efaedf5](#).

N-05 Missing Docstrings

Numerous declarations across `StableGold.sol`, including the `StableGold` contract, its state variables, its events, and many of its functions, lack NatSpec documentation. Complete docstrings improve readability, tooling output, and reviewer and integrator understanding.

Consider adding NatSpec comments that describe the purpose, parameters, and return values of at least the public and external interface.

Update: Resolved in [pull request #2](#) at commit [391873d](#).

N-06 Missing Named Parameters in Mappings

Multiple mappings, such as `admin`, `authority`, `custody`, `freezeList`, `kycStatus`, `onChainBBLimit`, `collectedPremiums`, and `collectedBBFees`, are declared without named key and value parameters. Since Solidity 0.8.18, named mapping parameters document intent inline and improve readability.

Consider adding named parameters to the mapping declarations.

Update: Acknowledged, not resolved. The team stated:

| We may add them later.

N-07 Missing Error Message in Revert Statement

The default branch of `redeem` calls `revert()` with no error message, so a caller that supplies an unsupported `_opt` value receives no indication of why the call failed.

Consider adding a descriptive error message or a custom error to this revert.

Update: Resolved in [pull request #2](#) at commit [e4e6986](#). The team stated:

| We have added the error message.

N-08 Non-Explicit Imports Are Used

The following imports in `StableGold.sol` bring in entire files rather than named symbols: `"./IDataFeed.sol"`, `"./AggregatorV3Interface.sol"`, `"./IERC7802.sol"`, `"./EIP3009.sol"`, and `"./SignatureChecker.sol"`. Global imports reduce clarity about which symbols are used and can introduce naming collisions, particularly in a file that already declares many contracts.

Consider using the named import syntax of the form `import {Symbol} from "./File.sol"`.

Update: Acknowledged, not resolved. The team stated:

| We will keep it as is.

N-09 `onChainBBLimit` Functions as a Lifetime Cap Rather Than a Periodic Rate Limit

In `onchainBuyBack`, each sale increments `onChainBBSpending[owner]` by `_amount` and then requires that the running total not exceed `onChainBBLimit[owner]`.

`onChainBBSpending` has a single write site, the increment above, and is never decremented, reset, or decayed, so `onChainBBLimit` functions as a cumulative lifetime cap

on the total an address may ever sell back on-chain, not as the per-period rate limit its name may suggest.

Once an address reaches its limit, it is permanently blocked from `onchainBuyBack` until a custodian raises `onChainBBLimit` for that address through `updateKYCStatus` or `updateKYCAddressBatch`.

Consider confirming that a lifetime cap is the intended semantics and, if a periodic rate limit was intended, tracking spending against a resettable window; otherwise, consider documenting that `onChainBBLimit` is a cumulative lifetime allowance.

Update: Acknowledged, not resolved. The team stated:

| *Lifetime cap is the intended.*

N-10 Ownership Transfer Is Single-Step and Can Irrecoverably Lose Control of the Contract

The contract uses a single-step ownership model inherited from the inlined `Ownable`. `transferOwnership` assigns the new owner immediately through `__transferOwnership`, with no requirement that the nominated account confirm the transfer. If ownership is transferred to an address that is mistyped, uncontrolled, or otherwise unable to act, the `owner`-only powers, namely pausing, `increaseSupply`, `reclaim`, `burnFrozenAssets`, and the feed, bridge, and feature setters, become permanently unrecoverable. The same single-step pattern governs the `DataFeedContract` owner.

Consider adopting a two-step ownership transfer, such as OpenZeppelin `Ownable2Step`, so that ownership moves to a new account only after that account explicitly accepts it.

Update: Acknowledged, not resolved. The team stated:

| *The ownership would be transfer on a multisig contract.*

N-11 View Functions `retrieveGoldPrices` and `retrieveOnChainGoldBBPrices` Return Prices Without a Staleness Check

The `retrieveGoldPrices` and `retrieveOnChainGoldBBPrices` view functions read `priceFeed.latestRoundData` and return derived gold, premium, and buyback prices

without the freshness and positivity checks that `buy` and `onchainBuyBack` apply inline. They can therefore return a stale price, or one derived from a non-positive answer, to any off-chain consumer, front end, or integrating contract that relies on them for display or quoting. The state-changing paths are unaffected, since they re-fetch and validate the price independently, so this is not a source of fund loss.

Consider applying the same staleness and positivity validation used in the state-changing paths within these view functions, or documenting them as unvalidated convenience views so that integrators do not treat them as trustworthy quote sources.

Update: Resolved in [pull request #2](#) at commit [42755b0](#). The team stated:

We have added checks as recommended.

N-12 `collectedBBFees` Tracks Notional Margin Instead of Segregated Funds

In `onchainBuyBack`, the fee accumulated into `collectedBBFees` represents stablecoins the contract withholds by paying the seller below spot, rather than an inflow the contract receives, because the buyback pays stablecoins out. `withdrawCollectedFees` later transfers `collectedBBFees[_token]` to a recipient and zeroes the counter, which only succeeds while the contract holds a sufficient balance. Because `withdrawERC20` can move the same shared token balance, including the liquidity required to pay future buybacks, without decrementing these counters, `collectedBBFees` and `collectedPremiums` are advisory figures that can diverge from the contract's actual holdings. For example, after `onchainBuyBack` raises `collectedBBFees[token]`, an owner `withdrawERC20` that sweeps that token's balance leaves the counter unchanged, so a later `withdrawCollectedFees` for the token reverts on the transfer despite the counter still reporting a positive amount.

Consider treating these counters as notional accounting only, and, if buyback payouts are intended to be guaranteed, reserving or segregating the payout liquidity from the owner-sweepable balance and reconciling the counters against actual token balances in any accounting model.

Update: Acknowledged, not resolved. The team stated:

Ordering is important, we first collect the fees by calling the `withdrawCollectedFees` function and then the `withdrawERC20` if needed. This will be documented.

N-13 Hardcoded EIP-712 Domain Name and Version Can Diverge from Token Metadata and Break Signed Transfers

`StableGold` verifies EIP-3009 signed transfers against an EIP-712 domain whose name and version are fixed at compile time as the private constants `DOMAIN_NAME = "Stablegold"` and `DOMAIN_VERSION = "1"`, consumed by `domainSeparator`. The ERC-20 token name is instead supplied at deployment through the `constructor` and returned by `name`, so the domain name and the on-chain token name are independent values that can differ. In particular, the hardcoded `"Stablegold"` differs in casing from the `StableGold` name used for the contract and symbol, so deploying with a `name` of `"StableGold"` would already produce such a divergence.

Any such mismatch causes off-chain tooling that builds the domain from `name` to produce a digest the contract rejects, so every signed transfer fails on-chain. The only exposed view, `domainSeparatorPublic`, returns only the separator hash and the chain id, which is insufficient for signers that need to reconstruct the individual domain fields.

Consider deriving the EIP-712 domain name from the ERC-20 `name` (and making the version explicit), or exposing getters for `DOMAIN_NAME` and `DOMAIN_VERSION`, or implementing an EIP-5267 `eip712Domain` view, so that signers can reconstruct the domain reliably; alternatively, consider constraining the constructor so the deployed `_name` matches the fixed domain name.

Update: Acknowledged, not resolved. The team stated:

| *We keep as is, for simplicity.*

N-14 Integrators Using `transferWithAuthorization` in Contract Flows Are Exposed to Front-Running

The token exposes two families of EIP-3009 signed-transfer entrypoints:

`transferWithAuthorization` with its `transferWithAuthorizationSign` variant, and `receiveWithAuthorization` with its `receiveWithAuthorizationSig` variant.

The `receive` variant requires the payee to equal `msg.sender`, so only the intended recipient can submit the authorization, while the `transfer` variant applies no such constraint and any party holding a valid signature can submit it.

Because a signed authorization becomes public once shared, whether handed to the integrator directly or routed through a relay, an integrator whose wrapper or deposit contract expects to be the party that consumes a `transferWithAuthorization` can be front-run: an attacker replays the same authorization directly against the token, the transfer settles and the nonce is consumed, but the wrapper's own bookkeeping, such as crediting the deposit, never executes and the funds may be stranded. Consumption emits an `AuthorizationUsed` event that integrators can monitor to detect a spent authorization. This is an inherent property of the generic EIP-3009 design rather than a defect in these entrypoints, and it does not affect a plain address-to-address payment.

Consider documenting for integrators that contract-destined signed transfers should use `receiveWithAuthorization` rather than `transferWithAuthorization`, so that consumption is bound to the recipient and the recipient's bookkeeping executes atomically with the transfer. This binds who may consume the authorization but does not by itself make the flow safe, since the recipient contract remains responsible for its own validation and bookkeeping.

Update: *Acknowledged, not resolved. The team stated:*

| *We will document it, thanks.*

N-15 Multiple Contract Declarations per File

`StableGold.sol` is a flattened file that declares many contracts, including `Context`, `Ownable`, `IERC20`, `ERC20`, `ERC20Burnable`, and `StableGold`, in a single source file. This makes the codebase harder to navigate and review and increases the risk of divergence from the canonical library sources.

Consider maintaining the contracts as separate imported files and flattening only as a build step when verification requires it.

Update: *Resolved in [pull request #2](#) at commit [8113fd1](#). The team stated:*

| *We have separated the files.*

N-16 Token Amount Calculation in `buy` Divides Before Multiplying

Within `buy`, the number of tokens to mint is computed as `noOfTokens = (amount / goldPricePremium) * 100000000`, performing the division before the multiplication. Because integer division truncates, dividing by `goldPricePremium` first discards the remainder before the subsequent scaling by `1e8`, so the minted amount loses precision relative to the intended value of `amount * 1e8 / goldPricePremium`. In the current configuration the resulting error is bounded below `1e-10` of a token per purchase and is therefore economically negligible, but the ordering remains a well-known precision antipattern that is trivially corrected.

Consider reordering the arithmetic to multiply before dividing, computing `noOfTokens` as `amount * 1e8 / goldPricePremium`, so that no precision is lost to premature truncation.

Update: Resolved in [pull request #2](#) at commit [b376552](#). The team stated:

| *We have fixed the reordering.*

N-17 Price and Reserve Feeds Use Redundant Chainlink-Shaped Interfaces

The gold price feed is read through `IDataFeed` (`priceFeed`) while the proof-of-reserve feed is read through `AggregatorV3Interface` (`chainReserveFeed`), yet both declare the identical Chainlink five-tuple `latestRoundData` and `getRoundData` signatures and a `decimals` accessor. This means two distinct interface types represent the same oracle shape for what is conceptually the same kind of Chainlink feed. The project documentation compounds the redundancy by describing `IDataFeed` as inheriting `AggregatorV3Interface`, which the interface does not do.

Consider consolidating both feeds onto a single Chainlink-compatible interface and correcting the documentation so that one oracle shape is represented by one type.

Update: Acknowledged, not resolved. The team stated:

| *We intentionally made the split of the two interfaces. We will document it.*

N-18 Proof-of-Reserve Can Be Enabled With a Zero Feed Address, Silently Falling Back to `maxSupply` Bound

`setChainReserveFeed` accepts `proofOfReserveEnabled` set to `true` together with a `chainReserveFeed` of `address(0)` without validation, and because every mint path guards with `chainReserveFeed == address(0) || proofOfReserveEnabled == false`, this configuration silently routes minting back to the `maxSupply`-only bound while `proofOfReserveEnabled` still reads `true`. Monitoring, or an operator relying on the reserve feed as the effective cap, may therefore believe issuance is bounded by live reserves when it is bounded only by `maxSupply`.

Consider rejecting a zero feed address whenever proof-of-reserve is enabled in the setter, and optionally probing `latestRoundData`, so that an enabled proof-of-reserve configuration cannot silently degrade to a `maxSupply`-only bound.

Update: Resolved in [pull request #2](#) at commit [5020df2](#). The team stated:

| *Thanks for the recommendation, we have added the control statement.*

Conclusion

The audited codebase implements StableGold, a gold-backed asset-reference token that issues an 18-decimal ERC-20 against accepted stablecoins at an oracle-provided gold price. On top of a standard ERC-20 core, the token layers oracle-driven pricing, role-based administration, KYC and freeze compliance controls, an optional proof-of-reserve supply bound, EIP-3009 signed transfers, and an ERC-7802 cross-chain mint and burn interface.

No critical-severity issues were identified. One high-severity issue and three medium-severity issues were identified during the engagement, which were subsequently resolved. The high-severity issue involved compliance freeze that can be bypassed because enforcement is confined to the transfer overrides and does not extend to burning, to the spender in `transferFrom`, or to minting, so a frozen balance is not reliably immobilized and the owner recovery paths that depend on it can be defeated. The medium- and low-severity findings cluster around a few recurring themes: the extension of that same compliance gap to the cross-chain mint and burn hooks, which apply neither the freeze list nor the KYC status and leave enforcement dependent on the external bridge, the pooling of multiple stablecoins into a single unsegregated reserve that a depeg can arbitrage, the use of the raw `IERC20` interface rather than a safe wrapper, which renders the core flows unusable for tokens such as USDT that omit boolean return values, the concentration of operational powers in the `admin` role, and configuration or availability pitfalls such as zero-valued oracle heartbeats, redemption that reverts until its bounds are configured, and burn paths that remain active while the contract is paused. A larger set of informational notes covers accounting counters that track notional rather than segregated funds, a hardcoded EIP-712 domain name that can diverge from the deployed token metadata, single-step ownership transfer, and a number of code-quality and documentation observations.

The codebase is organized around well-understood patterns, is deployed in a paused state that provides a useful configuration window before launch, and already includes several defensive checks such as oracle staleness guards and the optional proof-of-reserve bound. The system would nonetheless benefit from documenting its trust assumptions explicitly, in particular the operator-controlled nature of the backing invariant and the feed and bridge configuration that can be repointed instantly with no timelock, from consolidating its access-control and token-interaction patterns to reduce the operational surface, and from more consistent event coverage and docstrings to facilitate monitoring, integration, and future audits.

The StableGen team was responsive throughout the engagement and provided extensive documentation and relevant context.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.